五子棋开发文档

> 姓名：唐菁
> 班级：2406
> 学号：2024K8009970006

> 目录
> 1.quick-start
> 2.基本功能
> 3.从底层到上层的架构分解
> 4.其他的（未成功）的探索
> 5.反思和总结

quick-start:

1. 使用gcc编译人机对战，编译命令是：

```
gcc -std=c11 -O2 -Isrc `
  inc\ai\count_point_and_give_an_answer.c `
  inc\human_to_human\ban.c `
  inc\human_to_human\bound_checking_and_is_luozi.c `
  inc\human_to_human\different_types_of_chess.c `
  inc\human_to_human\look_at_surrounding.c `
  inc\human_to_human\place_chess.c `
  inc\main\main_for_ai_to_human.c `
  inc\main\show_board.c `
  inc\ai\kill_threat_proof.c`
  inc\ai\kill_threat_proof.h`
  -lm -o ai_vs_human.exe
```

是否加.exe对应着在windows下或者在linux下。
2. c-make

```
cmake_minimum_required(VERSION 3.16)

project(ai_vs_human C)

# 使用 C11
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)

# 优化等级（等价于 -O2）
set(CMAKE_C_FLAGS_RELEASE "-O2")

# 可执行文件
add_executable(ai_vs_human
    inc/ai/count_point_and_give_an_answer.c
    inc/ai/kill_threat_proof.c

    inc/human_to_human/ban.c
    inc/human_to_human/bound_checking_and_is_luozi.c
    inc/human_to_human/different_types_of_chess.c
    inc/human_to_human/look_at_surrounding.c
    inc/human_to_human/place_chess.c

    inc/main/main_for_ai_to_human.c
    inc/main/show_board.c
)

# 头文件搜索路径（等价于 -Isrc）
target_include_directories(ai_vs_human PRIVATE
    src
    inc
)

# 链接数学库（等价于 -lm）
target_link_libraries(ai_vs_human m)
```

基本功能

1. 悔棋功能：输入(-1,-1)即可悔棋
2. 棋盘复盘功能：结束后输出所有棋盘坐标

3. 判断胜负、判断禁手、判断是否不合规地落子
4. 人人对战：选择黑白棋之后输入坐标，即可在对应的棋盘上下棋
5. 人机对战：人类选择好执棋方后，和内置AI进行对战

从底层到上层的架构分解

最底层的函数

判断是否触碰边界

在所有需要判断的地方，都使用`in_bound`函数判断是否越界，防止出现突然的segmentation fault

判断周围的情况(look_at_surroundings.c)

判断的是下了这个位置之后新形成的棋型

1. 先返回一个方向的同色棋子数量：以向左为例，左边没有超过边界并且是同色棋子，那么说明左边就是自己的棋子，再向左移动，看看是否仍然是没有超过边界并且是同色棋子。这样可以遍历直到左侧的己方棋子到了尽头（即边界或者空或者对方）
2. 同时采集尽头的种类：block(对方or边界)/empty(空位置)
3. 再返回整个一条线的同色棋子数量：一共四个方向，上下，左右，两个斜方向。上下=上+下，以此类推

> 使用数据结构：结构体，耦合了长度、两头情况的综合结构体

判断棋型种类和禁手的函数(different_types_of_chess.c/ban.c)

此时底层函数起到了莫大的用处。
简单的连棋用这个就可以非常地好判断，直接判断是否满足连棋数量+两头要求即可（比如连活三：连棋3个，两头空空）
复杂一点的跳棋需要列出来各种跳棋具体的形状，根据这个点周围的连棋数量+两头要求也是可以推出来的。（比如xo空x这样的跳活三(x是自己的旧棋，o是自己的新棋)，只需要空这个位置的左边是连2且左空，位置的右边是连1且右空）
复杂的跳棋也可以通过递归来完成：比如当前长度不是四，并且填入后形成活四，那么就是活三了。

于是我们实现了活四、冲四、活三、眠三、眠二、活二的判断。

这样禁手判断基于棋型种类，也是容易的了：根据对应的下这个地方得到的棋型种类，是否有长连->双三->双四，就可以得出是否产生禁手棋型。

> 未来可以使用数组来归纳所有的棋型[目前用的是函数]

展示与存储棋盘

1. 用栈存储棋盘当前情况，(-1,-1)就退栈（悔棋），结束后将栈内的情况从头输出，从而得到棋盘回顾。
2. 心中的棋盘arrayForInnerBoardLayout用于记录当前每个位置棋子的种类，arrayForDisplayBoard用于在每一次心中棋盘更新的时候，都在对应的位置输出对应的点阵棋盘

人机对战实现

整个 AI 的决策过程可以概括为：

1. **危机预判**：在搜索前，先判断是否存在“必须防守”的紧急情况（对方冲四、活三等）。这个时候不递归了，立即堵住。
2. **评分筛选**：通过计算下了(x,y)的棋型，进而统计分数，快速找出当前局面下最有价值的几个候选点（Top-K），进入下一轮点。
3. **深度推演**：对筛选出的点进行递归搜索（我走这步 -> 对方走最优应手 -> 我再应对...），计算最终得分。
4. **动态防御**：如果在推演中发现对方威胁极大，会自动加深搜索层数，或者给防守棋增加额外的权重奖励。

二、 具体实现逻辑分层解析

1. 基础估值层

- **核心函数**：`give_point`
- **功能**：不进行递归，仅根据当前棋盘格局，计算某一点落下后形成的棋型分数。
- **实现细节**：
  - **棋型权重**：定义了 `POINT_OPEN_FOUR` (活四, 100万分), `POINT_SLEEP_FOUR` (冲四), `POINT_OPEN_THREE` (活三) 等常量。
  - **规则判定**：包含禁手判断（`is_banned_...`），如果是禁手直接返回负无穷。
  - **综合评分**：分数 = 活四数×权重 + 活三数×权重 + ...，从而算出来这一步的分数

2. 威胁感知与防御机制 (Threat & Defense Mechanics)

这是解决“贪攻不防守”的问题。

- **威胁检测**：
  - `max_immediate_threat` / `best_immediate_from_mask`：计算如果现在轮到对手下，对手能得到的最高分。如果这个分数很高（如达到活四级别），说明当前局面极度危险。
  - **动态阈值**（`ThresholdDyn`）：当检测到局势危险时，降低剪枝的门槛，允许 AI 搜索更多看似分数不高但能救命的防守棋。
- **强制防守逻辑 (Must-Block)**：
  - `collect_forced_block_points`：为了防止对手下 P → 我方防一手 → 对手再杀一手，我们无法抵御攻击的情况，系统需要尝试寻找是否有防守点能同时瓦解这些紧急的威胁。如果某些点不堵必输，则这些点被标记为“强制防守点”，跳过常规搜索直接决策。这是为了防止有些点当前分数不高(甚至进入不了Top-K),但是在两步之后的防守之中非常必要，就可能被漏掉。
  - `find_rule_adjudication_move`：**最高优先级的裁判**。如果我方能成五（直接赢）或者必须堵对方的成五，直接返回，不再浪费时间搜索；如果下一步我们能成活四或者对方成了活四，就一定要发展或者阻止。
- **防守奖励 (Defense Bonus)**：
  - 在递归函数 `best_value_symmetric` 中，计算 `bonus`。
  - **逻辑**：`bonus = (opp_now - opp_after) * 系数`。即：如果我这一步棋能让对手的威胁分数大幅下降，我就给这步棋加分。
  - **惩罚**：如果对手本来就要活四了，我走完这步对手还是能活四，则给予极大的惩罚（`MUST_BLOCK_PENALTY`），逼迫 AI 放弃这种无效防守。

3. 搜索策略层 (Search Strategy Layer)

这是 AI 的“大脑”，负责往后看几步。

- **递归评估**：`best_value_symmetric`->对称性递归
  - 每一层都以当前玩家最大化收益为目标进行递归，来模拟自己/对方玩家下一步的分数
  - **算法**：采用 最大化$Score = MyScore - (OpponentBestResponse \times Factor)$ 作为递归思路。
  - **对称性**：当前玩家寻找最大化（自己得分 - 对手反击得分）的招法。
  - **动态深度**：
    - 定义基础深度 `MAX_LAYER` (3层)。
    - **危机加深**：代码中 `if (opp_now >= POINT_OPEN_FOUR)`，如果发现局面危险，搜索

深度自动增加 `EXTRA_DEPTH_ON_THREAT`，防止被“算死”。

具体伪代码如下（详细逻辑）：

```
int best_value_symmetric(Board board, int layer, int max_layer) {
    if (layer >= max_layer || is_terminal(board)) {
        return evaluate(board); //层数过高直接返回
    }
    int best_score = -INF;
    for (Move move : collect_candidates(board)) {
        Board next = board.apply(move);
        int my_score = evaluate(next); //我本身在这一点的分
        int opp_score = best_value_symmetric(next, layer + 1, max_layer);
         //我下了这一点之后，模拟对手用相同算法的最大的分数
        int score = my_score - opp_score * FACTOR;  //得到我的分数和我考虑的对方的分数之间的差值
        if (score > best_score) best_score = score; //得到我下在这里之后的最好得分
    }
    // 动态加深
    if (opponent_threat_detected(board)) {
        return best_value_symmetric(board, layer, max_layer + EXTRA_DEPTH_ON_THREAT);
    }
    return best_score;
}
```

- **剪枝优化 (Pruning)**：
  - **Top-K 剪枝**：`collect_candidates` 和 `amount(layer)`。每层只选取静态分数最高的前 K 个点（如第一层19个，深层10个）进入递归，忽略烂棋。
  - **邻域剪枝**：`is_neighbored_5_5` / `3_3`。只搜索已有棋子周围的点，不搜索偏远空地。
  - **阈值剪枝**：如果某步棋的 `prune_score`（预估分）低于动态阈值 `ThresholdDyn`，直接丢弃，不进行深层递归。

4. 性能优化层 (Optimization Layer)

为了让复杂的逻辑跑得更快，提升效率。

- **置换表 (Transposition Table)**：
  - 使用 **Zobrist Hashing**（`compute_board_hash`，`xorshift64`）将棋盘状态映射为唯一的64位整数。
  - `tt_get` / `tt_put`：当遇到之前算过的局面时（例如通过不同顺序落子达到同一局

面），直接读取缓存的分数，跳过计算，从而增加速度。

- **掩码优化 (Masking)**：
  - 使用 `who_to_count` 数组记录哪些点是合法的候选点，只在这个上面进行更新，减少计算量。
  - 在递归中通过 `build_next_mask_from_base` 增量更新掩码，避免每层都遍历全盘 225 个点。指的是
    下一层的who_to_count直接拷贝父层的掩码作为基础；
    把刚刚落子的格子标记为不可再用（从 1 变 0）；
    然后后只在 (x,y) 周围那一小圈更新：如果某个原来不在候选里的空位，因为靠近这步新棋而变成“有邻居的空位”，就把它从 0 改成 1。
    这样：每一层维护的候选点集合都是“上一层 + 局部微调”，不用再对 15×15 整盘做“有无邻居”的判定，大幅减少时间复杂度。

综上，这个AI它将按以下步骤执行：
1步->2-3步->n步

1. **初始化与统计重置**：清空计数器，计算当前棋盘 Hash。
2. **规则裁决 (Rule Adjudication)**：
   - 棋盘上是否有“成五”。有 -> **直接赢。**
   - 对手是否有“成五”威胁。有 -> **必须堵。**
   - 自己是否有“活四/双三”。有 -> **直接下。**
   - 对手是否有“活四”。有 -> **必须堵。**
     这个是面对一步威胁所需要做的。
3. **强制防守检查 (Forced Block)**：
   - 如果规则裁决没结束，调用 `collect_forced_block_points`。
   - 模拟对手一步，我一步，对手再来一部，看我方是否只有唯一解。如果是，返回该解。
     这个是看2-3步。
4. **生成候选点 (Candidate Generation)**：
   - 找出所有空位中，周围有棋子的点。
   - 计算它们的静态分数 (`give_point`)。
5. **排序与筛选 (Top-K)**：
   - 对候选点排序，只取前 K 个。
6. **递归搜索 (Recursive Search)**：
   - 对这 K 个点加上强制防守检查的点，逐一调用 `best_value_symmetric`。
   - 在搜索中结合了防守奖励、置换表查找和剪枝。
     这个是看n步。

7. **决策**：
    - 返回得分最高的那个点作为最终落子。

其他（未成功）的尝试(AI_training文件夹)

当时（出于好奇和好胜心），想着用python训练一个神经网络->导入c/c++->进行决策，使用的是**alpha-zero-general**的架构。基本的训练架构是库里的本身的函数，但是游戏规则、神经网络是我这里完成的（虽然也有让AI优化过...），基本文件和原理介绍如下：

AlphaZero-General 的核心原理是：

> 用一个神经网络给出「直觉判断」（哪步好 + 形势好坏），
> 再用 **MCTS 搜索** 对这个直觉进行校正，
> 把校正后的结果反过来训练神经网络，
> 形成一个 **搜索 ↔ 学习 ↔ 更好搜索** 的闭环。

整体结构：三件核心组件

神经网络 (f_θ)

输入是棋盘状态 (s)，输出两样东西：

$$f_\theta(s) \to (p, v)$$

- **(p)**：policy
  → “我觉得哪些落子更有希望”（概率分布）
- **(v)**：value
  → “我觉得这个局面最后是赢还是输”（[-1,1]）,越大就是越容易赢。

这是类似于“直觉”的东西。

MCTS（蒙特卡洛树搜索）

作用：

> **用搜索去质疑和修正神经网络的直觉**

具体的步骤：

1. **选择 Selection**
    - 按公式（1）（这个公式后面进行解释！这里只需要知道，这是选择更多走哪个节点的

分，分越高走的越多。这里的P Q N都是旧值）：

$$a^* = \arg\max_a \left[Q(s,a) + c \cdot P(s,a) \frac{\sqrt{N(s)}}{1+N(s,a)} \right]$$

- 一直选到叶子节点.

2. **扩展 Expansion**
   - 叶子节点调用刚才提到的神经网络：

$$f_\theta(s_{leaf}) \to (p, v)$$

   - 初始化所有子动作的 P(s,a) = p(a)
3. **回传 Backup**
   - 将 v 沿路径回传，更新 Q(s,a), W(s,a), N(s,a)。
   - 从叶子节点回溯某个节点时，增加经过的次数N++,经过时将最后算得的v(价值)累加在累加价值W上，而后除以N得到平均价值Q。

重复以上步骤。

这个时候可以解释公式(1)了！公式1之中，Q是累加价值，累加价值越高更应该选；P是最开始神经网络输出的胜率，胜率越高，自然也更应该选。二者进行一个结合，得出当前的选择分数。从而得到修正之后的：**生成落子概率 π**

- 归一化搜索访问次数：

$$\pi(a \mid s) = \frac{N(s,a)^{1/\tau}}{\sum_b N(s,b)^{1/\tau}}$$

> π 是“搜索后策略”，即老师告诉学生“这一局面下应该走什么”，也即是你选的越多，说明这个节点真的很好。

选择动作

然后按 π 选择最应该走的地方，落子，生成下一个状态 (s_{t+1})，然后再进行搜索，更新π，重复直到终局

存储训练样本

每一步生成训练样本：

$$(s_t, \pi_t, z)$$

- (s_t)：棋盘状态
- (π_t)：MCTS 输出的落子分布
- (z)：终局胜负（+1 胜 / -1 负 / 0 和棋）

> 所有步骤都存起来，形成一整盘的数据集

更新神经网络

使用损失函数：

$$\mathcal{L}(\theta) = (z - v)^2 - \pi^T \log p + \lambda |\theta|^2$$

- **Value loss**：网络的每一步胜率学终局真正的胜率z
- **Policy loss**：网络的下一步先验直觉学 MCTS最后搜索得到的π

> 也就是说，让神经网络 **学会模仿MCT搜索**，同时学会胜负评估

最后

1. MCTS 越搜索越准，π 越可靠
2. 网络 v 越准，p 越像 π

> 网络越来越强 → 搜索更准 → 数据更好 → 网络更强

结果呈现及反思

人机对战的评分AI

总体效果还行，对于禁手的评判准确。然而关于局面的两个强制策略，其实都是评分函数有的时候对于某些强制情况无法进行正常识别之后的有些不得已的举动。后来反思：一个可能因为best_symmetric**模仿了对面的逻辑**也是自身分数减去对面的最大symmetric，但是对方可能不会这样想，许多稍差一点的对手可能想不到这么多，所以我这里就可能会痛失良机，而许多强劲的对手可能暗中进行深入布局了，和我所预料的下法也是不同的；第二个是，我的得分是my_point_here减去best_symmetric_of_the_opponent，**这两个得分很可能语义不一样**：因为黑白棋的先后手是不同的，所以这里也相对粗糙；第三个是，随之而来的，权值可能也调的比较粗糙，每一种棋手的权值最后竟然一样；第四个是思考层数太浅，应该在比赛的时候不管威胁程度如何，**都进行5层思考**，这样时间也是不会超过的（而不是威胁少的时候只有3层，威胁多才5层，这样其实会让我在安全的时候少了很多长期的布局）。

不过很满意这里的规则强制、强制搜索的几个，能够保证短期内的赢面；长期用对称地评分这个

逻辑也是无可厚非的，有一定高瞻远瞩性。

从代码的整洁性来说，前期的基础函数相当整洁，结构体等的运用也很美观！但是后期各种原因下，人机对战的代码不算简洁，也没有把不同功能块放在不同文件里，因为相互顺序依赖也很难再分开，所以这是以后需要注意的！有时间就应该提前将代码需要涉及的部分分开分好。

头文件声明+定义倒是很好的行为！

AI训练的失败尝试。

这个代码是查询后得到的，很多真正的训练的self-play积累数据是很多很多局，用很长时间的GPU进行训练，我这里self-play也就几十局，很具有随机性。并且这个模型一开始网络是随机的 → MCTS 也是在随机指导下走棋 → 盘面极其混乱：
很多局到很后面才出现第一个有威胁的棋型；
赢输更多是随机结果，不是“某一步好手”导致。
在这种数据上训练出来的网络，本质上学到的是“噪声分布”，很难把“好棋型”从终局里反推回来。

所以后面学到的真的是噪声（开机一个月，终局20秒）。

journal

25/12/25

圣诞节 写活三 if-else嵌套中 分类讨论好 希望后面不会有问题

25/12/27

发现有假装落子函数影响全局变量，重新定义结构体等，但是逻辑不变。（因为周五才学了结构体www)
完成禁手和判断逻辑，判断了棋子整个过程就不难办了

25/12/28

配了windows的C变量，主要觉得虚拟机和windows有两套环境比较安全
基本完成人人对战的流程（包括错误回退），唯一缺陷是禁手双三和双四不行，写了测试文件发现判断是对的

26/1/1

我想设置一个C语言的递归树，就是我下了a位置之后，有一个point()函数（已经写好，可以判断在当前棋谱的自己的分数和对方的分数），然后再在每个位置扫描一遍，算出对方下出来的最高分数（*后面改成了对方的最高递归分数*），两个以一个权值相减，得到本局的分数。然后这个分数和一个固定的和递归层数有关的f(n)相比，比这个大的继续递归（假设自己下了这个地方后的其他位置再扫描，然后算出下了各个位置的当前得分和对方可能拿的最高分，相减，得出每个地方可能的分值，和f(n)比较，然后加在原始位置的分上，然后继续递归），递归到最后，得出的最终分数最高的就是我想下的决策。

25/1/3

第一步遍历全局60s吗，那很有水平了->改成邻域计算

25/1/8

第一次下赢了朋友：Black(ai) wins the game!
the steps of the chess:
(7, 7) (6, 6) (6, 7) (5, 7) (7, 5) (7, 6) (5, 6) (4, 5) (6, 8) (8, 6) (8, 7) (7, 8) (9, 7) (10, 7) (10, 6) (8, 8) (9, 8) (9, 6) (8, 5) (6, 5) (6, 3) (9, 5) (5, 3) (4, 3) (4, 2) (3, 1) (5, 2) (6, 2) (5, 4) (5, 5) (4, 1) (7, 4) (5, 1) (5, 0) (3, 3) (2, 4) (4, 6) (4, 0) (3, 0) (2, 2) (2, 5) (4, 14) (3, 4) (5, 13) (0, 4) (6, 12) (3, 5) (3, 6) (9, 9) (4, 4) (9, 10) (9, 11) (8, 11) (6, 1) (10, 9) (7, 12) (11, 9) (8, 9) (8, 12) (7, 2) (11, 8) (12, 7) (11, 10) (12, 11) (11, 7) (11, 1) (11, 6)

26/1/14

效果一般，看看alpha-zero-general?

26/1/15

电脑风扇太吵了，放朋友服务器上跑?

26/1/16

朋友哈气他的核心占满了好几个，GPU也是不少。

26/1/22

端出来还是不如不要端出来。开始写之前的代码吧!

26/1/23

改进成对方也递归，比短视对方好些；调了一晚上权值，效果不佳啊。

26/1/24

加入规则决策+强制搜索+防守奖励，AI更能防守地好些，至少当天不要有低级错误错的太离谱。现在在我面前徐徐展开的有两门考试。

26/1/29

我和一友对战，我发现棋盘是错的，现改；他发现了segmentation fault，大家都有光明的未来。